

Diseño de API REST

1. ¿Qué es REST

- REST (**R**epresentational **S**tate **T**ransfer).
- Es un estilo arquitectónico para diseñar sistemas distribuidos, especialmente APIs web.
- No es un protocolo ni una tecnología concreta, sino una forma de organizar la comunicación entre cliente y servidor usando normalmente **HTTP**.
- Una API REST permite que una aplicación cliente, por ejemplo, una web, una app móvil o un frontend en React, pueda comunicarse con un backend para consultar, crear, modificar o eliminar recursos.

2. Idea principal de REST

- La idea central de REST es trabajar con **recursos**.
- Un recurso puede ser cualquier entidad del sistema:
 - Usuarios
 - Productos
 - Pedidos
 - Facturas
 - Cursos
 - Comentarios
 - Reservas
- Cada recurso se identifica mediante una **URL**. Por ejemplo:
 - /users
 - /users/15
 - /products
 - /products/8
 - /orders/25
- La API no debería estar centrada en acciones, sino en recursos.
 - Mejor: GET /users/15
 - Peor: GET /getUserById?id=15

3. Cliente y servidor

REST se basa en una separación clara entre:

- **Cliente**

- Es quien consume la API.
- Puede ser:
 - Frontend web
 - Aplicación móvil
 - Otro backend
 - Cliente de escritorio
 - Herramientas como Postman

- **Servidor**

- Es quien expone la API y gestiona los datos.
- Por ejemplo:
 - Backend en Node.js
 - Backend en Spring Boot
 - Backend en FastAPI
 - Backend en .NET
- El cliente no necesita saber cómo guarda los datos el servidor. El servidor puede usar MySQL, PostgreSQL, MongoDB, Oracle, ficheros, etc.

- **Recursos**

- Un **recurso** representa algo del dominio de la aplicación.
- Ejemplo en una aplicación de gimnasio:
 - [Usuario](#)
 - [Rutina](#)
 - [Entrenamiento](#)
 - [Ejercicio](#)
- Las URLs REST podrían ser:
 - [/users](#)
 - [/users/1](#)
 - [/routines](#)
 - [/routines/5](#)
 - [/workouts](#)
 - [/exercises](#)
- La URL identifica el recurso. La acción que se quiere realizar se indica mediante el método HTTP.

4. Métodos HTTP principales

REST utiliza los métodos HTTP para indicar qué operación se quiere hacer.

- **GET:**

- Se usa para consultar información.
- `GET /users` → Devuelve todos los usuarios.
- `GET /users/7` → Devuelve el usuario con ID 7.

- **POST:**

- Se usa normalmente para crear un nuevo recurso.

`POST /users`

Crea un usuario nuevo.

Cuerpo de la petición:

```
{  
  "name": "Ana",  
  "email": "ana@example.com"  
}
```

- **PUT:**

- Se usa para actualizar completamente un recurso.

`PUT /users/7`

Cuerpo:

```
{  
  "name": "Ana García",  
  "email": "ana.garcia@example.com"  
}
```

- Normalmente implica reemplazar el recurso completo.

- **PATCH:**

- Se usa para actualizar parcialmente un recurso.

`PATCH /users/7`

Cuerpo:

```
{  
  "email": "nuevo@email.com"  
}
```

- Solo modifica el campo email.

- **DELETE:**

- Se usa para eliminar un recurso.

`DELETE /users/7`

Elimina el usuario con ID 7.

5. Ejemplo básico de API REST

Supongamos una API para gestionar usuarios:

Operación	Método	URL
Obtener todos los usuarios	GET	/users
Obtener un usuario	GET	/users/{id}
Crear un usuario	POST	/users
Actualizar un usuario completo	PUT	/users/{id}
Actualizar parte de un usuario	PATCH	/users/{id}
Eliminar un usuario	DELETE	/users/{id}

Ejemplo:

[GET /users/3](#)

Respuesta:

```
{  
  "id": 3,  
  "name": "Pepe",  
  "email": "pepe@example.com"  
}
```

6. REST y HTTP

- REST suele implementarse sobre HTTP porque HTTP ya proporciona muchas de las piezas necesarias:
 - URLs
 - Métodos: GET, POST, PUT, PATCH, DELETE
 - Códigos de estado
 - Cabeceras
 - Cuerpo de petición y respuesta
 - Cacheo
 - Autenticación
- Aunque REST no obliga estrictamente a usar HTTP, en la práctica casi siempre se usa con HTTP.

7. Códigos de estado HTTP

Los códigos de estado indican el resultado de una petición.

Códigos 2xx: éxito	
200 OK	La operación se realizó correctamente. GET /users/1 Respuesta: 200 OK
201 Created	Se ha creado un recurso nuevo. POST /users Respuesta: 201 Created
204 No Content	La operación fue correcta, pero no se devuelve contenido. Muy común en DELETE. DELETE /users/1 Respuesta: 204 No Content
Códigos 3xx: redirección	
Menos habituales en APIs REST, pero existen.	
301 Moved Permanently	
302 Found	
304 Not Modified	se usa en escenarios de cacheo
Códigos 4xx: error del cliente	
400 Bad Request	La petición está mal formada. <u>Ejemplo</u> : falta un campo obligatorio. <pre>{ "error": "The email field is required" }</pre>
401 Unauthorized	El usuario no está autenticado. 401 Unauthorized <u>Ejemplo</u> : falta el token.
403 Forbidden	El usuario está autenticado, pero no tiene permisos. 403 Forbidden <u>Ejemplo</u> : un usuario normal intenta borrar otro usuario.
404 Not Found	El recurso no existe. GET /users/9999 Respuesta: 404 Not Found
409 Conflict	Hay un conflicto con el estado actual del sistema. <u>Ejemplo</u> : intentar crear un usuario con un email que ya existe. 409 Conflict
422 Unprocessable Entity	La petición tiene formato correcto, pero los datos no pasan la validación. <u>Ejemplo</u> : <pre>{ "email": "no-es-un-email" }</pre>

Códigos 5xx: error del servidor	
500 Internal Server Error	Error inesperado del servidor
503 Service Unavailable	El servicio no está disponible temporalmente

8. Formato de intercambio de datos

- El formato más común en APIs REST es **JSON**.

- Ejemplo de respuesta:

```
{  
  "id": 1,  
  "name": "Laura",  
  "email": "laura@example.com"  
}
```

- Antes también se usaba mucho XML, pero hoy JSON es el estándar de facto.

9. Cabeceras HTTP importantes

- Las cabeceras permiten enviar metainformación.

Cabecera	Descripción	Ejemplo
Content-Type	Indica el formato del cuerpo enviado.	Content-Type: application/json
Accept	Indica qué formato espera recibir el cliente.	Accept: application/json
Authorization	Se usa para enviar credenciales o tokens.	Authorization: Bearer ey.JhbGciOiJIUzI1NiIs...
Location	Se suele usar al crear un recurso.	HTTP/1.1 201 Created Location: /users/25 Indica la URL del nuevo recurso creado.

10. Ejemplo completo de petición y respuesta

Crear usuario

Petición:	Respuesta:
POST /users HTTP/1.1 Host: api.example.com Content-Type: application/json Authorization: Bearer token123 { "name": "Lucía", "email": "lucia@example.com"	HTTP/1.1 201 Created Content-Type: application/json Location: /users/10 { "id": 10, "name": "Lucía", "email": "lucia@example.com"

}	}
---	---

11. Principios de REST

REST se basa en varias restricciones o principios arquitectónicos:

1. Cliente-servidor:

- El cliente y el servidor tienen responsabilidades separadas.
- El cliente se encarga de la interfaz y la experiencia de usuario.
- El servidor se encarga de la lógica de negocio, datos y seguridad.

2. Sin estado (stateless)

- Cada petición debe contener toda la información necesaria para ser procesada.
- El servidor no debe depender de una sesión interna para recordar el estado del cliente.
- Ejemplo correcto:

`GET /profile`

`Authorization: Bearer token123`

Cada petición incluye el token.

Ejemplo menos RESTful:

`GET /profile`

`Cookie: SESSION_ID=abc123`

- Esto no significa que no se puedan usar cookies, pero REST favorece que cada petición sea autocontenida.

3. Cacheable:

- Las respuestas deben poder indicar si se pueden cachear o no.
- Ejemplo:
`Cache-Control: max-age=3600`
- Esto permite mejorar rendimiento y reducir carga del servidor.

4. Interfaz uniforme

- Las APIs REST deben tener una forma consistente de interactuar con los recursos.
- Por ejemplo:
`GET /products`
`GET /products/1`
`POST /products`
`PUT /products/1`
`DELETE /products/1`
- La estructura es predecible.

5. Sistema por capas

- El cliente no necesita saber si está hablando directamente con el servidor final o con una capa intermedia.
- Puede haber:

Cliente
API Gateway
Balanceador de carga
Servidor backend
Base de datos

6. Código bajo demanda

- Es una restricción opcional. Permite que el servidor envíe código ejecutable al cliente, por ejemplo, JavaScript.
- En APIs REST modernas no suele ser lo más relevante.

12. Recomendaciones de diseño de URLs

- Una buena API REST usa nombres de recursos en plural.

Mejor	Peor
/users	/getUsers
/products	/createProduct
/orders	/deleteOrder

- La acción ya la indica el método HTTP.
- Ejemplos correctos:

GET /users
GET /users/12
POST /users
PUT /users/12
PATCH /users/12
DELETE /users/1

- Relaciones entre recursos

Si un usuario tiene pedidos: GET /users/12/orders Devuelve los pedidos del usuario 12.	Si quieres consultar un pedido concreto de ese usuario: GET /users/12/orders/5
---	--

- Evitar URLs demasiado profundas

Buena opción	Demasiado complejo
	GET /companies/3/departments/8/users/12/orders/5/items/9

GET /users/12/orders	En esos casos suele ser mejor simplificar: GET /order-items/9 o usar filtros: GET /orders?companyId=3&departmentId=8&userId=12
----------------------	--

13. Parámetros en REST

Hay tres formas habituales de enviar datos en una petición REST.

Tipos	Descripción	Ejemplo
Path parameters	- Forman parte de la URL. - Se usa cuando identificas un recurso concreto.	GET /users/15 Aquí 15 es un parámetro de ruta.
Query parameters	- Se indican después de ? - Para filtros, búsquedas, ordenación y paginación	GET /users?role=admin&active=true GET /products?category=books GET /products?minPrice=10&maxPrice=50 GET /users?sort=name GET /orders?status=pending
Body	- Para enviar datos complejos - normalmente con POST, PUT o PATCH.	POST /users Content-Type: application/json <pre>{ "name": "María", "email": "maria@example.com" }</pre>

14. Filtros, ordenación y paginación

Una API REST bien diseñada debería permitir consultar listas de forma controlada.

Filtros	GET /products?category=electronics GET /orders?status=paid GET /users?active=true
Ordenación	GET /products?sort=price GET /products?sort=-price - Una convención habitual: - sort=price ordenar ascendente - sort=-price ordenar descendente - También puede hacerse así: GET /products?sortBy=price&order=desc

Paginación	- Si hay muchos resultados, no conviene devolverlos todos. Ejemplo: <code>GET /products?page=2&limit=20</code> Respuesta: <pre>{ "data": [{ "id": 21, "name": "Product 21" }], "pagination": { "page": 2, "limit": 20, "totalItems": 135, "totalPages": 7 } }</pre> - También se puede usar paginación por cursor: <code>GET /products?cursor=abc123&limit=20</code> - Paginación por cursor → mejor para grandes volúmenes de datos
-------------------	--

15. Idempotencia

- Significado: ejecutar varias veces la misma operación produce el mismo resultado final.

Método	¿Es idempotente?	Observaciones
GET	Sí	- Consultar varias veces no cambia el estado. <code>GET /users/1</code>
PUT	Sí	- Si se envían siempre los mismos datos, el resultado final es el mismo. <code>PUT /users/1</code>
DELETE	Suele serlo	- Si se borra una vez, queda borrado. - Si se intenta borrar otra vez, el estado final sigue siendo que no existe. <code>DELETE /users/1</code>
POST	Normalmente no	- Se usa para crear recursos nuevos o ejecutar operaciones no idempotentes. - Si se hace la misma petición varias veces, se podrían crear varios usuarios:

POST /users

16. Diferencia entre POST, PUT y PATCH

Ejemplo: Si tenemos en la base de datos:

```
{
  "id": 5,
  "name": "Pedro",
  "email": "pedro@example.com",
  "active": true
}
```

Método	Descripción	Ejemplo
POST	- Crear un recurso nuevo o ejecutar una operación. POST /users - El backend decide el ID.	
PUT	- Reemplazar completamente un recurso existente. PUT /users/5 - Se envía la información completa del usuario.	<pre>{ "name": "Pedro López", "email": "pedro@example.com", "active": true }</pre>
PATCH	- Modificar parcialmente un recurso. PATCH /users/5 - Se envías solo los campos que quieres cambiar.	<pre>{ "name": "Pedro López" }</pre>

17. Errores en APIs REST

- Una buena API REST debe devolver errores consistentes.
- Ejemplo de error:

```
{
  "error": {
    "code": "USER_NOT_FOUND",
    "message": "User not found",
    "details": {
      "userId": 99
    }
  }
}
```

- Otro ejemplo de validación:

```
{
```

```
"error": {  
  "code": "VALIDATION_ERROR",  
  "message": "Invalid input data",  
  "fields": {  
    "email": "Invalid email format",  
    "password": "Password must have at least 8 characters"  
  }  
}
```

- Mucho mejor que devolver simplemente: [Error](#)

18. REST y CRUD

- REST encaja muy bien con operaciones CRUD:

Create
Read
Update
Delete

- Correspondencia típica:

CRUD	HTTP	Ejemplo
Create	POST	POST /products
Read	GET	GET /products/1
Update	PUT / PATCH	PUT /products/1
Delete	DELETE	DELETE /products/1

- REST se usa mucho para representar operaciones más complejas.
 - Por ejemplo:

Acción	Operación REST
Confirmar un pedido	POST /orders/25/confirmation
Cancelar una reserva:	POST /reservations/10/cancellation
Pagar una factura:	POST /invoices/7/payment

- **Idea clave:** modelar la acción como un recurso o subrecurso cuando sea posible.

Mala opción: [POST /confirmOrder](#)

Buena opción: [POST /orders/25/confirmation](#)

19. REST en Node.js con Express

```
const express = require("express");
const app = express();
```

```
app.use(express.json());
```

```
let users = [
  { id: 1, name: "Ana" },
  { id: 2, name: "Luis" }
];
```

```
////////////////////////////////////
app.get("/users", (req, res) => {
  res.json(users);
});
```

```
////////////////////////////////////
app.get("/users/:id", (req, res) => {
  const user = users.find(u => u.id === Number(req.params.id));

  if (!user) {
    return res.status(404).json({
      error: "User not found"
    });
  }

  res.json(user);
});
```

```
////////////////////////////////////
app.post("/users", (req, res) => {
  const newUser = {
    id: users.length + 1,
    name: req.body.name
  };

  users.push(newUser);

  res.status(201).json(newUser);
});
```

```
////////////////////////////////////
app.patch("/users/:id", (req, res) => {
```

```
const user = users.find(u => u.id === Number(req.params.id));

if (!user) {
  return res.status(404).json({
    error: "User not found"
  });
}

if (req.body.name !== undefined) {
  user.name = req.body.name;
}

res.json(user);
});
////////////////////////////////////

app.delete("/users/:id", (req, res) => {
  users = users.filter(u => u.id !== Number(req.params.id));
  res.status(204).send();
});

////////////////////////////////////

app.listen(3000, () => {
  console.log("API running on port 3000");
});
```

20. REST en Spring Boot

```
@RestController
@RequestMapping("/users")
public class UserController {

  private final List<User> users = new ArrayList<>();

  //////////////////////////////////////

  @GetMapping
  public List<User> getUsers() {
    return users;
  }

  //////////////////////////////////////

  @GetMapping("/{id}")
  public ResponseEntity<User> getUser(@PathVariable Long id) {
    return users.stream()
      .filter(user -> user.getId().equals(id))
      .findFirst()
      .map(ResponseEntity::ok)
      .orElse(ResponseEntity.notFound().build());
  }
}
```

```
}
```

```
////////////////////////////////////
```

```
@PostMapping
```

```
public ResponseEntity<User> createUser(@RequestBody User user) {  
    users.add(user);  
    return ResponseEntity.status(HttpStatus.CREATED).body(user);  
}
```

```
////////////////////////////////////
```

```
@PatchMapping("/{id}")
```

```
public ResponseEntity<User> updateUser(  
    @PathVariable Long id,  
    @RequestBody User updatedUser  
) {  
    for (User user : users) {  
        if (user.getId().equals(id)) {  
            user.setName(updatedUser.getName());  
            return ResponseEntity.ok(user);  
        }  
    }  
  
    return ResponseEntity.notFound().build();  
}
```

```
////////////////////////////////////
```

```
@DeleteMapping("/{id}")
```

```
public ResponseEntity<Void> deleteUser(@PathVariable Long id) {  
    users.removeIf(user -> user.getId().equals(id));  
    return ResponseEntity.noContent().build();  
}  
}
```

21. DTOs

- En APIs REST es habitual usar Data Transfer Object (DTOs).
- Se emplea para no exponer directamente las entidades de la base de datos.

Entidad interna:	DTO de respuesta:
<pre>public class User { private Long id; private String name; private String email; private String password; }</pre>	<pre>public class UserResponseDTO { private Long id; private String name; private String email; }</pre>

22. REST y frontend

- Un frontend puede consumir una API REST usando fetch, axios o librerías similares.
- Ejemplos con JavaScript:

<pre>// DTO para recibir usuarios class UserDTO { constructor(id, name, email) { this.id = id; this.name = name; this.email = email; } }</pre>	<pre>// DTO para crear usuarios class CreateUserDTO { constructor(name, email) { this.name = name; this.email = email; } }</pre>
<pre>// GET /users ////////////////////////////////////// async function getUsers() { const response = await fetch("https://api.example.com/users"); const data = await response.json(); return data.map(user => new UserDTO(user.id, user.name, user.email)); } // POST /users ////////////////////////////////////// async function createUser() { const dto = new CreateUserDTO("Ana", "ana@example.com"); const response = await fetch("https://api.example.com/users", {</pre>	



```
method: "POST",

headers: {
  "Content-Type": "application/json"
},

body: JSON.stringify(dto)
}
);

const data = await response.json();

return new UserDTO(
  data.id,
  data.name,
  data.email
);
}
```